PROYECTO DE EFICIENCIA ENERGÉTICA DEL AULA

Integrantes: Vicente Martínez, David Martínez, Víctor Bleda

Curso: Inteligencia Artificial y Big Data 2025 - 2026

Índice

Introducción.....1
Contexto del Problema.....2
Objetivos del Sistema.....2
Estructura de carpetas.....2
Análisis Exploratorio de Datos (EDA).....3
Analisis de Correlacion de sensores.....3
Agrupación por horas.....3
Análisis de sensores de ventanas y puerta.....4
Origen de Datos y Arquitectura Medallion.....5
Capa Bronze (RAW).....5
Capa Silver (Datos Limpios y Normalizados).....5
Capa Gold (Datos Agregados Para IA).....5
Modelo de Machine Learning.....6
Predicción de Temperatura (Modelo Lineal).....6
Algoritmo de Etiquetado.....6
Evaluación del Modelo.....6
Red Neuronal.....7
Preparación de Datos.....7
Transformación Cíclica.....7
División y Estandarización de los Datos.....7
Problema de Desbalanceo.....8
Búsqueda del modelo Óptimo.....8
Modelo Ganador.....8
Api REST Con Flask.....12
Endpoints Disponibles.....12
Flujo de Predicción.....12
Visualización y Explotación del Dashboard.....13
Monitorización de Aperturas.....13
Evolución Térmica.....14
Indicadores de Estado Rápido.....14
Análisis Meteorológico.....15
Conclusiones.....15

Introducción

Contexto del Problema

El presente documento detalla el desarrollo de una solución basada en IoT e Inteligencia Artificial para la optimización del consumo energético en un entorno educativo. El proyecto surge de la necesidad de detectar situaciones de ineficiencia, específicamente el uso de sistemas de climatización (calefacción) mientras existan ventanas o puertas abiertas.

Objetivos del Sistema

El objetivo principal es diseñar un modelo capaz de predecir situaciones de derroche energético con una hora de antelación. Para lograrlo, el sistema debe:

- **Centralizar la telemetría:** Recopilar datos de temperatura, humedad, presión y estado de puerta/ventanas mediante sensores Zigbee y Shelly.
- **Procesar y Refinar (Arquitectura de Medallón):** Organizar los datos en capas (Bronze, Silver, Gold) para asegurar su calidad y trazabilidad.
- **Inferir Estados Críticos:** Utilizar modelos de *Machine Learning* lineal para determinar la temperatura de la calefacción y su estado de encendido.
- **Predecir el Riesgo:** Implementar una Red Neuronal de clasificación binaria para anticipar el derroche en la hora siguiente.
- **Visualizar y Monitorizar:** Desplegar un *dashboard* interactivo en Grafana para la supervisión en tiempo real.

Estructura de carpetas

```
energy-efficiency-medallion/
├── dashboard/      # Paneles de visualización (Grafana/JSON)
├── data/           # CSV's en niveles Bronze, Silver y Gold
├── docker/         # Base de datos PostgreSQL
├── docs/           # Informe técnico y manuales del proyecto
├── models/         # Modelos entrenados (.pkl para ML y .keras para IA)
├── notebooks/      # Notebooks de Jupyter (EDA, entrenamiento modelos y api)
├── scripts/        # Scripts para la generación de csv's en bronze, silver y gold
├── slides/         # Presentación para la defensa del proyecto
├── .gitignore      # Archivos excluidos del repositorio
└── README.md       # Documentación principal del repositorio
```

Para ejecutar el proyecto se deberá lanzar el archivo api.py en la carpeta nootebooks

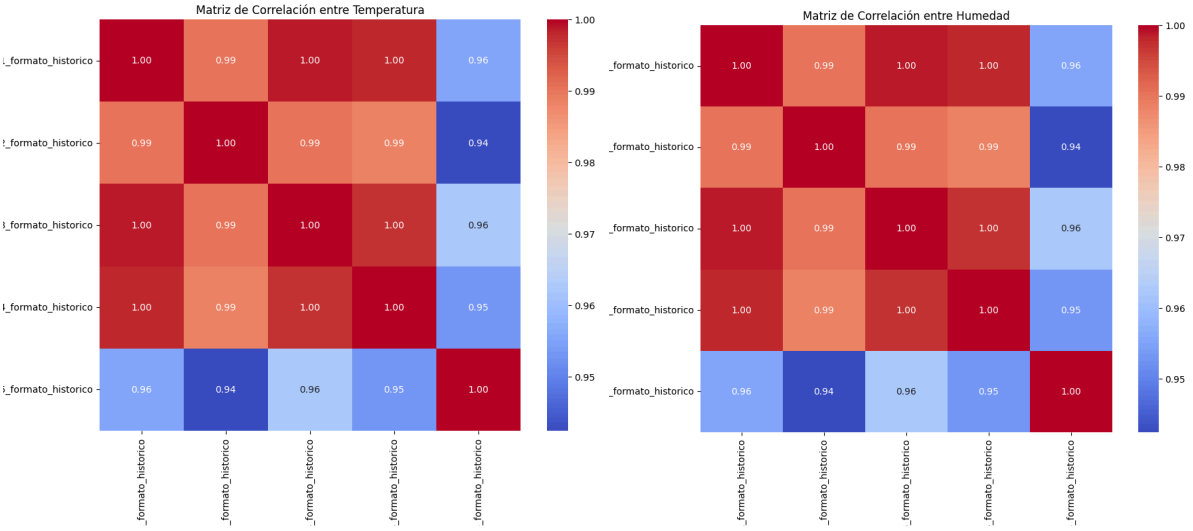
Análisis Exploratorio de Datos (EDA)

Como paso previo a la construcción del modelo predictivo, hemos realizado un Análisis Exploratorio de Datos para entender el comportamiento de las variables físicas del aula y definir la mejor estrategia para agrupar los datos por horas.

Analisis de Correlacion de sensores

Para evaluar la necesidad de mantener todos los sensores desplegados en el aula, hemos calculado las matrices de correlación lineal (Pearson) para las mediciones de temperatura y humedad de los 5 sensores disponibles.

- **Resultados:** Tal y como se observa en los mapas de calor, existe una correlación casi perfecta entre la mayoría de los dispositivos. Los sensores 1, 3 y 4 presentan un índice de 1.00, mientras que el resto oscilan entre 0.94 y 0.99.



- **Decisión:** Como hemos comprobado que los sensores marcan prácticamente lo mismo, no tiene sentido guardar datos repetidos. Hemos decidido quedarnos únicamente con los datos del **sensor 3** como referencia de temperatura y humedad para toda el aula, y descartar las columnas del resto de sensores.

Agrupación por horas

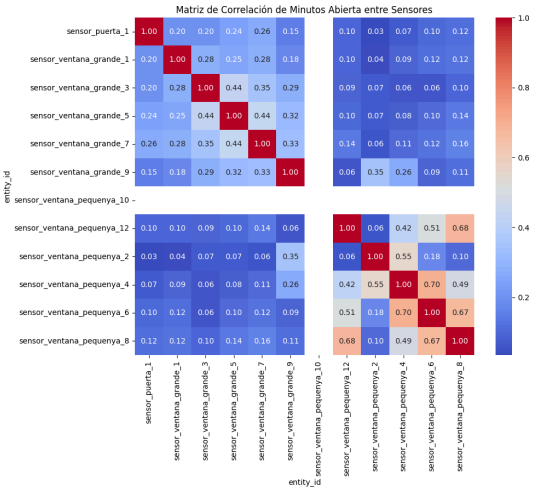
Como vamos a predecir a una hora vista, los datos por segundo no nos sirven: lo agrupamos todo en bloques de 1 hora.

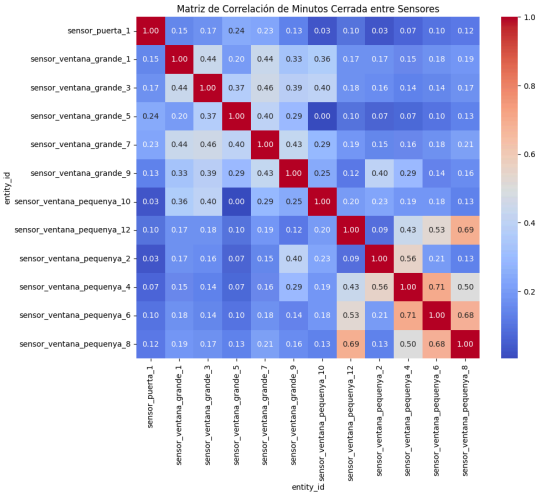
- **Media (el promedio):** La usamos para casi todo. Así sacamos la tendencia general de esa hora.
- **Moda (el más repetido):** La usamos **solo** para la dirección del viento. Como son puntos cardinales (Norte, Sur...) y no se pueden promediar, simplemente cogemos la dirección que más ha soplado en esa hora.

Análisis de sensores de ventanas y puerta

Respecto a los sensores de las ventanas hemos decidido no utilizar los valores de temperatura y humedad, ya que no eran necesarios, por ello usaremos solo el sensor que nos indica cuando están abiertas o cerradas las ventanas.

- **Correlación:** Los sensores de las ventanas no están prácticamente correlacionados por lo que vamos a necesitar usarlos todos para tener los máximos valores posibles.





- **Datos de los sensores:** Respecto a cómo hemos usado los datos, hemos cogido las horas en las que se abrían los sensores y lo hemos pasado a cuántos minutos por hora están abiertas o cerradas las ventanas, o puerta.
  De esta forma es mucho más sencillo calcular el derroche.

- **Derroche:** El derroche lo hemos establecido en un 45% ya que consideramos que es lo más equilibrado, lo que quiere decir que si en una hora el tiempo máximo que están abiertas todas las ventanas supera el 45% hay derroche.

- **Organización de sensores:** Para que los datos tuvieran sentido tuvimos que hacer unos ajustes en los datos, es decir, ventana tenemos 12, de las cuales hay 6 grandes y 6 pequeñas, hemos eliminado la 11 porque el sensor estaba roto, las ventanas grandes los minutos los multiplicamos por 2 ya que son más grandes, al igual que la puerta también multiplicamos los minutos por 2.

Origen de Datos y Arquitectura Medallion

Para garantizar la calidad, trazabilidad y correcta preparación de los datos antes de alimentar nuestros modelos, se ha implementado una arquitectura de datos en tres capas (Medallion Architecture). La fuente principal es una base de datos PostgreSQL, que actúa como repositorio centralizado para la telemetría de los sensores integrados en Home Assistant y la información de servicios externos (como Met.no y Sun).

Capa Bronze (RAW)

Esta capa actúa como el lago de datos principal. Contiene la ingesta histórica y en bruto extraída directamente de la base de datos PostgreSQL.

- **Contenido:** Lecturas originales de los sensores (temperatura, humedad, presión, consumo eléctrico) y estados (puertas/ventanas abiertas o cerradas) sin ningún tipo de alteración en formato csv.
- **Objetivo:** Mantener un historial inmutable que sirva como fuente única de la verdad en caso de necesitar reprocesar la información desde cero.

Capa Silver (Datos Limpios y Normalizados)

En esta fase, los datos crudos se someten a un proceso de depuración (EDA y limpieza).

- **Procesamiento:** Se tratan los valores nulos y se estandarizan los formatos de fecha y hora. Además, tras realizar un análisis de correlación, se identifican y descartan los sensores redundantes para optimizar el conjunto de datos.
- **Objetivo:** Obtener un *dataset* estructurado y fiable, manteniendo la granularidad original de los eventos, listo para cruces y agregaciones más complejas.

Capa Gold (Datos Agregados Para IA)

Diseñada específicamente para el entrenamiento de los modelos predictivos.

- **Agregación horaria:** Se agrupan los datos por horas, calculando medias de temperatura/humedad y sumatorios.
- **Ingeniería de variables (*Features*):** Se integran los datos externos (meteorología, posición del sol) y se infiere el estado de la calefacción mediante un modelo base de Machine Learning.
- **Creación del Target:** Se define la condición de "derroche" (calefacción encendida + ventanas abiertas > X minutos) y se desplaza temporalmente para predecir el derroche en la **hora siguiente**.

Modelo de Machine Learning

Para poder entrenar nuestra Red Neuronal de clasificación y predecir el riesgo de derroche energético, necesitábamos contar con una variable fundamental: el estado de la calefacción. Sin embargo, la telemetría directa del aula no contaba con un sensor físico que registrara este estado específico, lo que nos impedía etiquetar los datos de la capa Gold correctamente.

Predicción de Temperatura (Modelo Lineal)

Para resolverlo, recurrimos a una estrategia de inferencia cruzando los datos del **Sensor 3** (nuestra referencia del aula) y el **Sensor 5** (ubicado en la sala de la calefacción).

- **Entrenamiento:** Entrenamos un modelo de Machine Learning lineal con sklearn utilizando el histórico de datos en el que disponíamos de lecturas simultáneas de ambos sensores.
- **Predicción:** Una vez ajustado el modelo, le pasamos los datos históricos del Sensor 3 (aula) para predecir, hora a hora, cuál era la temperatura equivalente en la sala del Sensor 5 (calefacción).

Algoritmo de Etiquetado

Con la serie temporal de temperaturas predichas para el Sensor 5, el siguiente paso fue traducir esa temperatura al estado real de la calefacción (Encendida/Apagada). Para ello, implementamos un algoritmo basado en la programación automática real del sistema de climatización del centro.

El algoritmo evalúa la temperatura predicha cruzándose con el día de la semana y la franja horaria para generar la columna booleana **CE**:

- **Franjas diurnas principales (06:59 - 16:00 y 18:02 - 20:40, L-V):**
  - Si la temperatura predicha es **< 21.9°C**, (CE = 1).
  - Si la temperatura supera los **22.0°C**, (CE = 0).

Evaluación del Modelo

- **MAE (Error Absoluto Medio): 0.54 °C.** En promedio, nuestras predicciones fallan por apenas medio grado.
- **RMSE (Error Cuadrático Medio): 0.75 °C.**
- **R^2 (Coeficiente de Determinación): 0.93.** Esto significa que el 93% de la variabilidad de la temperatura en el cuarto de la calefacción es explicada.

Red Neuronal

Una vez consolidada nuestra capa Gold con todos los datos necesarios y nuestra variable objetivo generada, el siguiente paso fue diseñar, entrenar y optimizar una Red Neuronal capaz de predecir el derroche energético con una hora de antelación.

Preparación de Datos

El primer paso para que el modelo aprenda es separar la información en dos bloques:

- **Las variables predictoras (X):** Todas las columnas de nuestro dataset (temperatura, humedad, meteorología, estado de la calefacción actual, etc.) que el modelo usará para buscar patrones.
- **La variable objetivo (Y / Target):** Nuestra columna final que indica si en la siguiente hora habrá un derroche energético o no. Es lo que queremos que la red neuronal aprenda a predecir.

Transformación Cíclica

Uno de los grandes retos con las variables temporales o direccionales (como la hora del día, el mes, el día de la semana o la dirección del viento) es que, matemáticamente, los números extremos están muy lejos, cuando en la realidad están juntos. Por ejemplo, para un ordenador, la hora 23 y la hora `0` tienen una diferencia de 23 unidades, aunque en la realidad solo están separadas por una hora.

Para solucionar esto, aplicamos una **transformación matemática usando el Seno y el Coseno**. Esto proyecta estos valores en un círculo, permitiendo que la red neuronal entienda la continuidad espacial y temporal a la perfección (sabe que diciembre está pegado a enero, o que las 23:00h están pegadas a las 00:00h).

División y Estandarización de los Datos

Antes de entrenar, dividimos nuestro conjunto de datos en dos partes (train_test_split):

- **80% para Entrenamiento (Train):** Los apuntes con los que la red neuronal estudia.
- **20% para Pruebas (Test):** El examen final que la red nunca ha visto, para comprobar si realmente ha aprendido o solo ha memorizado.

Además, aplicamos un StandardScaler para **estandarizar** los datos. Esto pone todas las variables (X) en la misma escala. Si no lo hiciéramos, variables con números muy grandes (como los grados del azimut del sol) aplastarían la importancia de variables con números pequeños.

Problema de Desbalanceo

Nuestra primera prueba se realizó con una red pequeña 32, 16, 1 y 300 épocas. El resultado fue muy malo. ¿El motivo? **El desbalanceo de clases**.

En la vida real, el aula pasa mucho más tiempo funcionando "bien" que en situación de "derroche". Si el 98% del tiempo no hay derroche, un modelo inútil que siempre diga "Todo está bien" acertará el 98% de las veces, pero fallará en su verdadero propósito.

La Solución:

1. **Pesos (Class Weights):** Aplicamos la regla pesos = {0: 1.0, 1: 32.0}. Le dijimos a la red neuronal que fallar al detectar un derroche (clase 1) es **32 veces más grave** que equivocarse en una hora normal. Básicamente, forzamos a la red a prestar muchísima más atención a los casos raros de ineficiencia.
2. **Métricas Reales:** Dejamos de mirar la "Precisión (Accuracy) general" y empezamos a evaluar la red usando métricas más exigentes que miden los falsos positivos y falsos negativos (Recall, F1-Score).
3. **Reducción del Learning Rate:** Bajamos drásticamente la "tasa de aprendizaje" a 0,0001. Esto hace que la red dé pasos mucho más pequeños y cautelosos al aprender, evitando que pase de largo de la solución óptima.

Búsqueda del modelo Óptimo

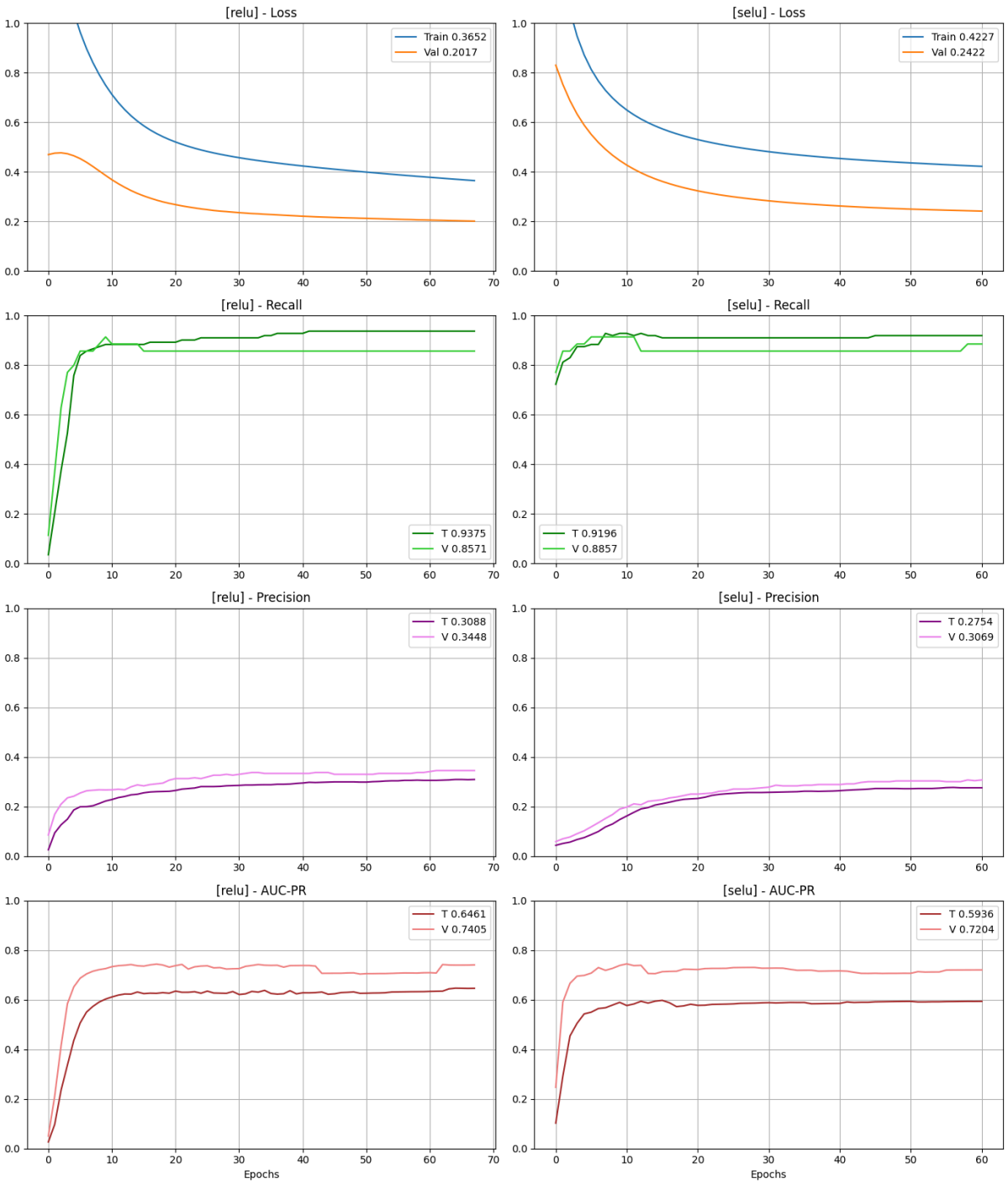
Con el problema del desbalanceo solucionado, lanzamos una red de entrenamientos comparando diferentes arquitecturas de red durante 1000 épocas. Probamos redes con funciones de activación *ReLU* y *SELU*, y con distintas profundidades

```
redes = [
    [[32, 16, 1], "relu", 1000],
    [[32, 16, 1], "selu", 1000],
    [[128, 64, 32, 1], "relu", 1000],
    [[128, 64, 32, 1], "selu", 1000],
    [[256, 128, 64, 32, 1], "relu", 1000],
    [[256, 128, 64, 32, 1], "selu", 1000]
]
```

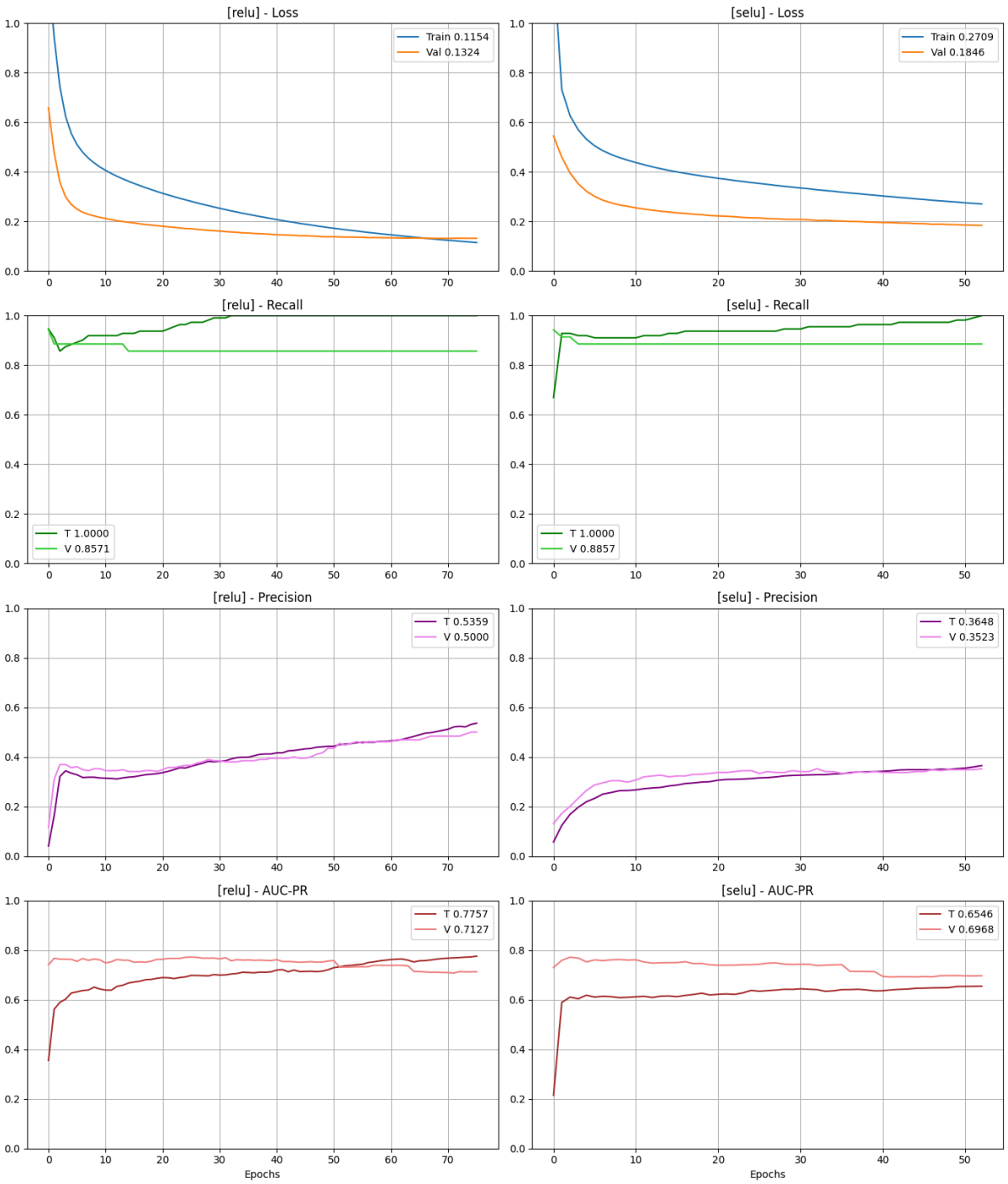
Modelo Ganador

- **Estructura:** Capas ocultas de **128 -> 64 -> 32 -> 1** neuronas
- **Activación: ReLU**.
- **Épocas:** 1000 épocas.

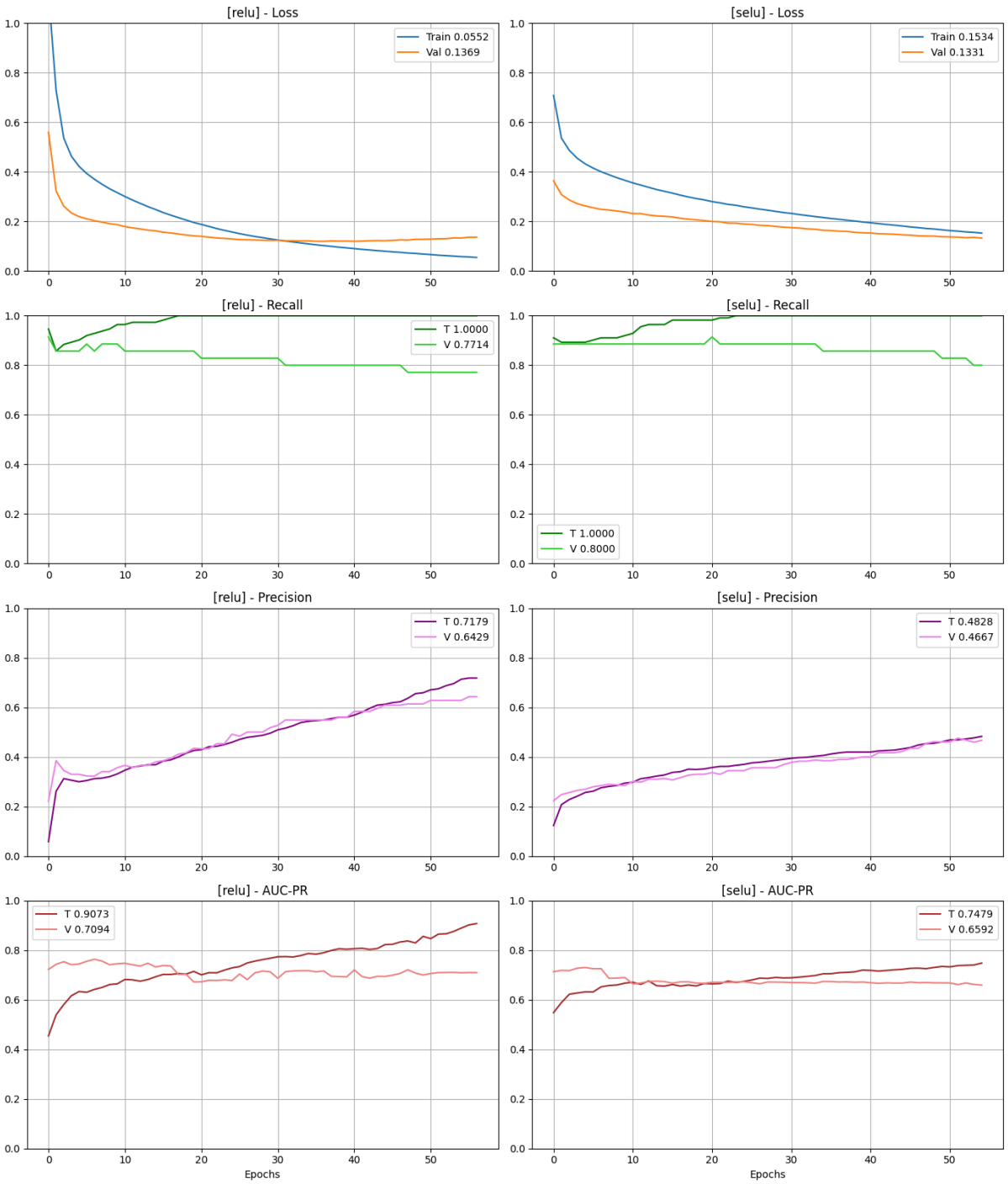
Arquitectura: [32, 16, 1] (Class Weights)



Arquitectura: [128, 64, 32, 1] (Class Weights)



Arquitectura: [256, 128, 64, 32, 1] (Class Weights)



Api REST Con Flask

Una vez que nuestra Red Neuronal fue entrenada y validada, necesitábamos una forma de comunicarnos con ella en tiempo real. Para lograrlo, desarrollamos una API REST utilizando **Flask**, un framework ligero de Python.

Endpoints Disponibles

La API expone dos rutas principales de comunicación (*endpoints*):

- **GET / (Home):** Un endpoint sencillo que devuelve una interfaz web básica (index.html) para comprobar que la API está viva y funcionando.
- **POST /predict (El Motor de Inferencia):** Está diseñado para recibir un paquete de datos JSON, procesarlos y devolver una predicción inmediata.

Flujo de Predicción

Cuando la API recibe una petición HTTP con los datos meteorológicos y del aula en formato JSON, ejecuta un proceso de transformación idéntico al que hicimos durante el entrenamiento, garantizando que el modelo "entienda" la información:

1. **Recepción y Conversión:** El JSON entrante se transforma en un *DataFrame* de Pandas.
2. **Transformación Cíclica (Seno y Coseno):** Al igual que en el entrenamiento, aplicamos la transformación trigonométrica a las variables temporales y direccionales (mes, dia_de_la_semana, hora, mislata_viento_dir, sol_azimut).
3. **Alineación de Columnas:** Reordenamos el *DataFrame* generado usando el archivo orden_columnas.pkl. Si el modelo aprendió que la temperatura era la primera columna y la humedad la segunda, esta línea de código fuerza a que los datos nuevos respeten esa estructura.
4. **Estandarización:** Pasamos los datos por nuestro scaler para ponerlos en la misma escala numérica con la que el modelo fue entrenado.
5. **Inferencia y Respuesta:** Los datos ya limpios y procesados se introducen en la Red Neuronal. El modelo devuelve un valor numérico (la predicción de riesgo de derroche), que la API empaqueta de nuevo en un JSON y se lo devuelve al sistema solicitante.

Visualización y Explotación del Dashboard

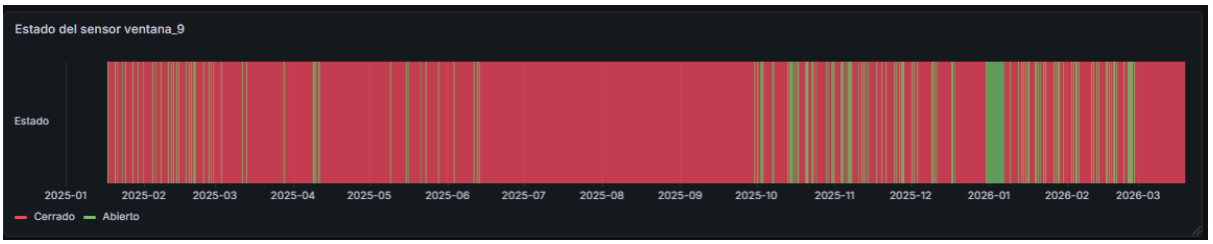
Para la monitorización de los datos almacenados en PostgreSQL, hemos desarrollado un panel de control interactivo en Grafana. El diseño se ha centrado en la utilidad práctica, permitiendo que la información sea fácil de interpretar sin necesidad de analizar bases de datos complejas.

Paneles de Control y Justificación de Uso

Monitorización de Aperturas

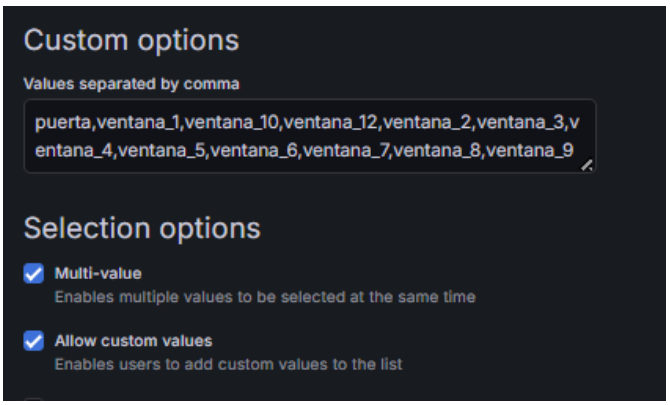
Para el seguimiento de los accesos, hemos utilizado una combinación de visualización de estados y conteo de eventos:

- **Línea de Estados :** Muestra de forma visual cuándo un sensor ha estado abierto (**verde**) o cerrado (**rojo**). Hemos simplificado la interfaz eliminando textos innecesarios para que el código de colores sea el protagonista.



- **Contador de Aperturas:** Hemos implementado un cálculo que solo suma una unidad en el momento exacto en que se abre la puerta o ventana, evitando errores de conteo por duración, esto nos muestra el número de veces que el sensor ha cambiado de 0 a 1 en el tiempo determinado.

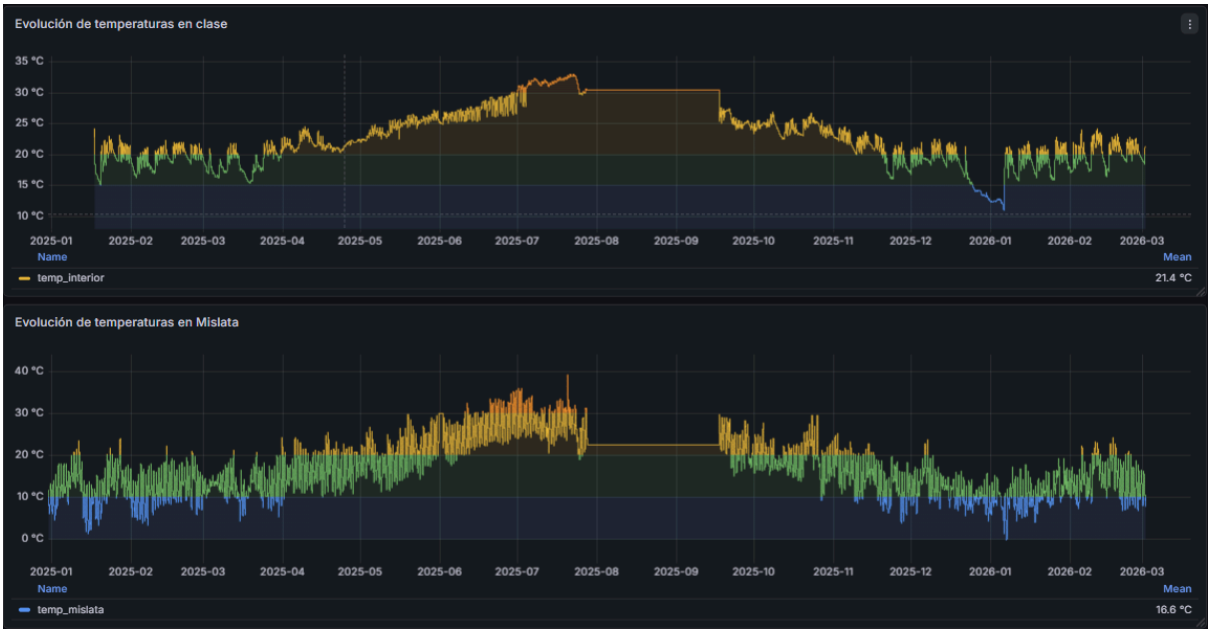
- **Uso de Variables:** Para no saturar el dashboard con muchos paneles distintos, hemos creado la variable **$sensor**. Esto permite que, con un solo clic en el menú superior, el usuario pueda conmutar entre la puerta o cualquiera de las ventanas. El dashboard se actualiza al instante, facilitando la comparación y ahorrando espacio.



Evolución Térmica

Es el componente central para entender cómo varía el clima en el aula y compararlo con el exterior:

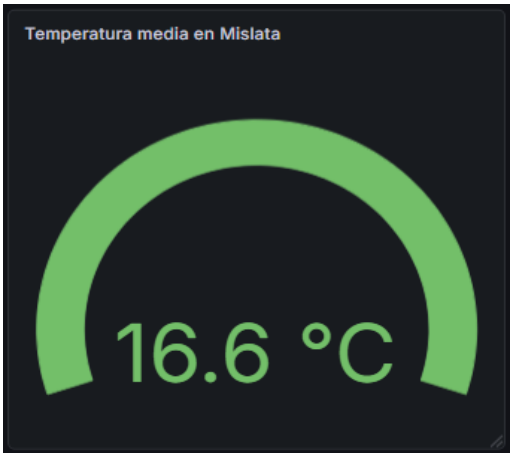
- **Gráficas de Series Temporales:** Muestran el histórico de temperatura de la clase frente a la de Mislata, hemos decidido hacerlo en dos cuadros distintos, ya que, para su comparación pensamos que es lo mejor.
- **Justificación de Colores:** Hemos aplicado un degradado que actúa como indicador visual. La línea cambia de azul (frío) a verde (confort) y naranja (calor).



Indicadores de Estado Rápido

Son los paneles semicirculares que muestran el valor medio de temperatura.

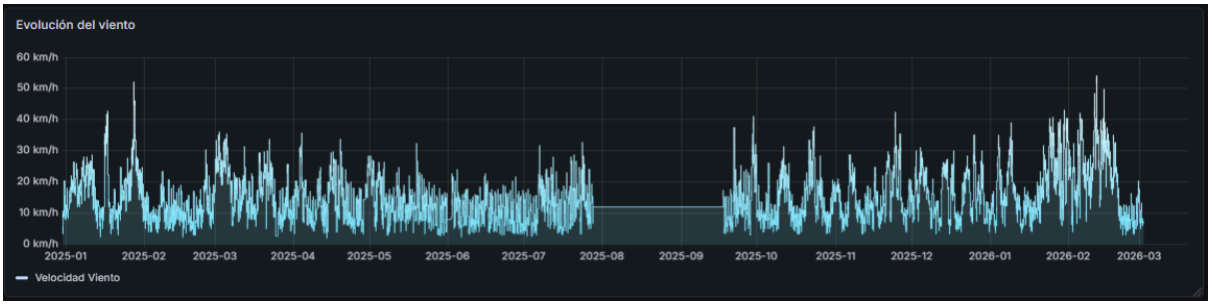
- **Justificación:** Mientras que las gráficas muestran la evolución histórica, los *Gauges* sirven para dar una respuesta inmediata. Nos permiten ver la diferencia térmica real entre el interior y el exterior de un solo vistazo, funcionando como un resumen rápido del estado térmico actual.



Análisis Meteorológico

Hemos añadido un panel específico para la velocidad del viento en el exterior.

- **Justificación:** Nos permite justificar caídas bruscas de temperatura interior: si detectamos una bajada de grados que coincide con ráfagas de viento fuertes en la gráfica, podemos confirmar que la pérdida de calor se debe a la entrada de aire y no a un fallo en la climatización.



Conclusiones

El desarrollo de este sistema de monitorización y predicción permite extraer conclusiones fundamentales sobre la digitalización de la eficiencia energética en entornos educativos:

- **Robustez de la Arquitectura de Datos:** La implementación de la **Arquitectura Medallion** ha demostrado ser esencial. El paso de los datos brutos (**Bronze**) a un dataset refinado (**Silver**) y finalmente a uno agregado por horas con ingeniería de variables (**Gold**), garantiza que el modelo de IA trabaje con información de alta calidad, eliminando redundancias detectadas en el EDA.
- **Eficacia del Soft-Sensing:** Ante la falta de un sensor físico en la caldera, la creación de un **modelo lineal de inferencia térmica** (con un R^2 de 0,93) ha validado que es posible monitorizar sistemas complejos mediante la correlación de sensores existentes, reduciendo la necesidad de hardware adicional.
- **Superación del Desbalanceo:** El entrenamiento de la **Red Neuronal** confirmó que la precisión general (*Accuracy*) no es una métrica válida en entornos reales de derroche, donde los casos críticos son minoritarios. El uso de **Class Weights (1:32)** y una tasa de aprendizaje reducida han permitido obtener un modelo capaz de identificar situaciones de riesgo con una hora de antelación de forma efectiva.
- **Cuantificación del Derroche:** La definición de un **umbral del 45%** (basado en la apertura ponderada de ventanas grandes, pequeñas y puerta) establece un criterio objetivo y técnico para la gestión energética. Esto permite transformar percepciones subjetivas sobre el frío o el calor en métricas accionables.
- **Interoperabilidad y Visualización:** La integración de una **API REST con Flask** y un dashboard en **Grafana** cierra el ciclo del proyecto, convirtiendo las predicciones matemáticas en información visual y accesible en tiempo real para la toma de decisiones.

En conclusión, el proyecto cumple con el objetivo de **anticipar la ineficiencia energética**, demostrando que la combinación de IoT y Deep Learning es una herramienta potente para la sostenibilidad de los centros docentes, permitiendo una gestión proactiva que evita el gasto innecesario antes de que ocurra.